

---

# Reinforcement Learning Report

---

Rainbow: Combining Improvements in Deep  
Reinforcement Learning

Boyenval Thibaut

---

February 2025



# Table of Contents

|                                  |    |
|----------------------------------|----|
| Introduction .....               | 1  |
| Notations .....                  | 2  |
| 1 Deep Q-Networks .....          | 3  |
| 2 Extensions .....               | 3  |
| 2.1 Double Q-learning .....      | 4  |
| 2.2 Prioritized replay .....     | 5  |
| 2.3 Dueling Networks .....       | 7  |
| 2.4 Multi-step learning .....    | 8  |
| 2.5 Distributional RL .....      | 9  |
| 2.6 Noisy Nets .....             | 11 |
| 3 Combination .....              | 12 |
| 4 Experimental Analysis .....    | 15 |
| 5 Implementation .....           | 17 |
| 6 Improvements for Rainbow ..... | 18 |
| References .....                 | 19 |

## Introduction

This report aims to present and summarize the concepts introduced in the article "*Rainbow: Combining Improvements in Deep Reinforcement Learning*." It is divided into six sections, five of which focus on the article and its content. One section stands apart, providing details about the accompanying notebook. This notebook is designed to implement some of the methods discussed in the report and to illustrate the key concepts.

# Notations

Reinforcement learning involves an agent interacting with an environment through actions and rewards. The agent's objective is to act in a way that maximizes the rewards obtained as a result of its actions.

In the remainder of this report, we will adopt the same notations and context as presented in the article "Rainbow":

The interaction between the agent and its environment occurs at discrete time steps, denoted by  $t = 0, 1, 2, \dots$ . At each time step  $t$ , the agent receives an observation  $\mathcal{S}_t$  from the environment, selects an action  $\mathcal{A}_t$ , and subsequently receives a reward  $R_{t+1}$ , a discount factor  $\gamma_{t+1}$ , and the next state  $\mathcal{S}_{t+1}$ . This process is commonly described using the framework of a Markov Decision Process (MDP).

An MDP is formally defined as a tuple  $(\mathcal{S}, \mathcal{A}, T, r, \gamma)$ , where:

- $\mathcal{S}$  is a finite set of states.
- $\mathcal{A}$  is a finite set of actions.
- $T(s' | s, a) = P[\mathcal{S}_{t+1} = s' | \mathcal{S}_t = s, \mathcal{A}_t = a]$  represents the stochastic transition function.
- $r(s, a) = E[R_{t+1} | \mathcal{S}_t = s, \mathcal{A}_t = a]$  is the reward function.
- $\gamma \in [0, 1]$  is the discount factor.

In our context, we consider episodic MDPs where the discount factor is constant at  $\gamma$ , except at the termination of an episode where  $\gamma = 0$ .

Actions are chosen based on a policy  $\pi$ , which defines a probability distribution over actions for each state. Starting from a given state  $\mathcal{S}_t$  at time  $t$ , the agent aims to maximize the discounted return  $G_t$ , defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma_t^{(k)} R_{t+k+1}$$

where the discount factor for a reward  $k$  steps into the future is the product of all previous discounts:

$$\gamma_t^{(k)} = \prod_{i=1}^k \gamma_{t+i}.$$

The agent's goal is to maximize the expected discounted return by optimizing its policy. The policy may be learned directly, or it can be derived as a function of other learned quantities. In value-based reinforcement learning, the agent estimates:

- The state value function:  $v^\pi(s) = E[G_t | \mathcal{S}_t = s]$ .
- The state-action value function:  $q^\pi(s, a) = E[G_t | \mathcal{S}_t = s, \mathcal{A}_t = a]$ .

# 1. Deep Q-Networks

The Deep Q-Network (DQN) algorithm is a reinforcement learning method designed to address the challenges of approximating the Q-function for high-dimensional state spaces. Introduced by Mnih et al. [2], DQN leverages deep neural networks to approximate the action-value function  $q(s, a)$ , enabling effective decision-making in complex environments.

DQN aims to find an optimal policy  $\pi^*$  by approximating the Q-function defined as:

$$q^*(s, a) = E_{s' \sim T, r \sim R} \left[ r + \gamma \max_{a'} q^*(s', a') \mid s, a \right]$$

where:

- $s$  and  $s'$ : current and next states,
- $a$  and  $a'$ : current and next actions

Instead of explicitly storing  $q(s, a)$  in a table (as in classical Q-learning), DQN uses a neural network  $q(s, a; \theta)$  parameterized by  $\theta$ .

It introduces an **experience replay buffer** to store tuples  $(s, a, r, s')$  from past interactions. During training, mini-batches are sampled uniformly from this buffer to:

- Break correlations between consecutive samples.
- Improve sample efficiency by reusing experiences.

To stabilize training, DQN employs a separate **target network**  $q(s, a; \bar{\theta})$  with parameters  $\bar{\theta}$ , which are periodically updated to match  $\theta$ . The target Q-value is computed as:

$$y = r + \gamma \max_{a'} q(s', a'; \bar{\theta})$$

The mean squared error between the predicted Q-values and the target values is minimized:

$$\mathcal{L}(\theta) = E_{(s, a, r, s') \sim \text{ReplayBuffer}} \left[ (y - q(s, a; \theta))^2 \right]$$

The optimization is performed using RMSprop algorithm on mini-batches sampled uniformly from the experience replay which contains the last million transitions.

## 2. Extensions

Since the introduction of DQN, numerous papers have sought to improve its performance by modifying the original algorithm. These successful attempts encouraged the authors of the paper [1] to combine various enhancements that addressed independent issues while maintaining the same underlying algorithm. While previous works had attempted to combine two or three improvements, [1] stands out by demonstrating the performance of the algorithm with six enhancements to DQN:

- Double Q-Learning

- Prioritized replay
- Dueling networks
- Multi-step learning
- Distributional RL
- Noisy Nets

## 2.1 Double Q-Learning

In traditional Q-learning, the action-value function  $q(s, a)$  is updated using the following Bellman equation:

$$q(s, a) \leftarrow q(s, a) + \alpha \left[ r + \gamma \max_{a'} q(s', a') - q(s, a) \right]$$

Here,  $\max_{a'} q(s', a')$  serves two purposes:

1. **Action selection:** Determines the action  $a'$  that maximizes the Q-value at the next state  $s'$ .
2. **Value estimation:** Estimates the value of the chosen action  $a'$ .

This dual role introduces a bias known as overestimation bias, where the Q-values are systematically overestimated. This occurs because the same function is used for both selecting and evaluating the action, leading to an optimistic estimation, especially when the Q-function is poorly approximated during training.

Double Q-learning, introduced by Hasselt et al. [3], mitigates overestimation bias by decoupling the action selection and value evaluation steps. It achieves this by maintaining two separate Q-functions,  $q_A$  and  $q_B$ , which are updated alternately. The update rule is:

$$q_A(s, a) \leftarrow q_A(s, a) + \alpha \left[ r + \gamma q_B \left( s', \arg \max_{a'} q_A(s', a') \right) - q_A(s, a) \right]$$

Alternatively, if  $q_B$  is updated, the rule is:

$$q_B(s, a) \leftarrow q_B(s, a) + \alpha \left[ r + \gamma q_A \left( s', \arg \max_{a'} q_B(s', a') \right) - q_B(s, a) \right]$$

Key steps:

1. **Action selection:** The action  $a'$  is selected using **arg max** on one Q-function (e.g.,  $q_A$ ).
2. **Value evaluation:** The value of the selected action is computed using the other Q-function (e.g.,  $q_B$ ).

By splitting these roles across two separate estimators, Double Q-learning reduces the overestimation bias.

In the Deep Q-Network framework, the introduction of Double Q-learning follows the same principle but adapts it for neural network-based Q-function approximation. Specifically:

- Instead of maintaining two separate neural networks for  $q_A$  and  $q_B$ , Double DQN uses the primary Q-network ( $q_{\text{online}}$ ) and the target network ( $q_{\text{target}}$ ) to decouple action selection and value estimation.

The target value  $y$  in Double DQN is computed as:

$$y = r + \gamma q_{\text{target}}\left(s', \arg \max_a q_{\text{online}}(s', a'; \theta); \bar{\theta}\right)$$

Here:

- $q_{\text{online}}(s', a'; \theta)$ : The online Q-network, responsible for selecting the action  $a'$ .
- $q_{\text{target}}(s', a'; \bar{\theta})$ : The target Q-network, responsible for evaluating the value of  $a'$ .

By decoupling action selection and evaluation, Double Q-learning prevents the same network from reinforcing its optimistic biases during training. This reduced bias leads to more accurate value estimates, resulting in smoother convergence and more stable learning dynamics.

The integration of Double Q-learning into DQN is seamless, as it leverages the existing target network used for stabilizing training. The additional computational cost is minimal, as it only requires a single forward pass through the target network for evaluating  $q_{\text{target}}$ .

## 2.2 Prioritized replay

Experience replay is a critical component of Deep Q-Networks, designed to improve sample efficiency and reduce the correlation between consecutive experiences. However, the standard experience replay samples experiences uniformly, which may not be optimal. Not all transitions in the replay buffer contribute equally to learning:

- Some transitions provide more significant learning signals (e.g., when the current Q-value is far from the target Q-value).
- Uniform sampling treats all experiences equally, which can slow down convergence by over-sampling redundant or less informative transitions.

Prioritized Experience Replay (Schaul et al., 2016) [4] addresses this limitation by prioritizing transitions based on their learning potential, enabling the agent to focus on more informative experiences.

The main idea is to assign a priority  $p_i$  to each transition  $i$  in the replay buffer, where  $p_i$  reflects the importance of the transition. Transitions with higher priorities are sampled more frequently during training.

The priority  $p_i$  is typically proportional to the **Temporal Difference error**:

$$\delta_i = |y_i - q(s_i, a_i; \theta)|$$

where:

- $y_i = r_i + \gamma \max_{a'} q(s'_i, a'; \theta^-)$ : The target Q-value.
- $q(s_i, a_i; \theta)$ : The predicted Q-value from the online network.

A high TD error indicates that the network is learning substantially from the corresponding transition, justifying its higher priority.

Transitions are sampled from the replay buffer using a probability distribution proportional to their priorities:

$$P(i) = \frac{p_i^\alpha}{\sum_j p_j^\alpha}$$

Here:

- $\alpha \in [0,1]$ : A hyperparameter controlling the degree of prioritization ( $\alpha = 0$  reduces to uniform sampling).
- $p_i$ : The priority of transition  $i$ .

Prioritized sampling introduces a bias because it over-samples certain transitions. To correct this, importance sampling weights are applied during gradient updates:

$$w_i = \left( \frac{1}{N} \cdot \frac{1}{P(i)} \right)^\beta$$

where:

- $w_i$ : The IS weight for transition  $i$ .
- $N$ : The size of the replay buffer.
- $\beta \in [0,1]$ : A hyperparameter that anneals over time to reduce the bias as learning progresses.

The TD error updates are scaled by  $w_i$ , ensuring unbiased gradient updates in expectation.

Types of prioritization:

1. **Proportional Prioritization**: The priority  $p_i$  is directly proportional to  $|\delta_i|$  (with a small constant  $\epsilon > 0$  added to ensure numerical stability):

$$p_i = |\delta_i| + \epsilon$$

2. **Rank-Based Prioritization**: Transitions are ranked by their TD errors, and priorities are assigned based on their ranks. This approach is less sensitive to outliers but requires sorting the buffer.

In the DQN framework, integrating prioritized replay involves:

1. Modifying the replay buffer to store priorities for each transition.
2. Using prioritized sampling during mini-batch selection.
3. Adjusting gradient updates with IS weights to correct sampling bias.

The additional computational cost is manageable, involving priority computation (TD error) during updates and sampling using a priority queue or segment tree data structure for efficiency.

## 2.3 Dueling networks

In standard Q-learning and DQN, the Q-function is represented as:

$$q(s, a) = E \left[ r + \gamma \max_{a'} q(s', a') \mid s, a \right]$$

This formulation directly estimates the action-value  $q(s, a)$  for every action  $a$  at a given state  $s$ . However, in many states, the choice of action has little impact on the Q-value. For example:

- In terminal states, all actions may yield similar rewards.
- In states where one action is clearly superior, the relative values of other actions might be less significant.

This redundancy wastes representational capacity in the Q-network, as it computes  $q(s, a)$  for all actions independently, even when state-dependent values dominate.

Dueling Network Architectures (Wang et al., 2016) [5] address this inefficiency by separately estimating:

1. **State-value function  $V(s)$ :** The intrinsic value of being in a state  $s$ , irrespective of the action.
2. **Advantage function  $A(s, a)$ :** The relative importance of taking a specific action  $a$  compared to others at state  $s$ .

The Q-function is decomposed as:

$$q(s, a) = V(s) + A(s, a)$$

To enforce identifiability (i.e., ensuring the decomposition is unique), the advantage function is normalized, typically using a mean subtraction:

$$Q(s, a) = V(s) + \left( A(s, a) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a') \right)$$

where  $|\mathcal{A}|$  is the number of possible actions.

This architecture involves three key components:



1. A **shared convolutional or fully connected feature extractor** that processes the input state  $s$ .
2. Two **separate streams**: one for estimating  $V(s)$  and another for  $A(s, a)$ .
3. A **combination layer** that computes  $q(s, a)$  from  $V(s)$  and  $A(s, a)$ .

The training objective remains the same as in DQN, minimizing the Temporal Difference (TD) loss:

$$\mathcal{L}(\theta) = E_{(s,a,r,s') \sim \text{ReplayBuffer}} \left[ (y - q(s, a; \theta))^2 \right]$$

The main difference lies in the architecture, as the Q-function is now computed via  $V(s)$  and  $A(s, a)$ , rather than being directly estimated.

The dueling network architecture is seamlessly integrated into the DQN framework:

1. **Feature Extraction**: The shared layers are the same as standard DQN.
2. **Parallel Streams**: Two fully connected layers are added—one for  $V(s)$  and one for  $A(s, a)$ .
3. **Combination Layer**:  $q(s, a)$  is computed from  $V(s)$  and  $A(s, a)$  using the mean-subtracted formula.

The computational overhead is minimal compared to standard DQN, while the architectural changes significantly enhance performance.

## 2.4 Multi-step learning

Traditional reinforcement learning, algorithms like Q-learning or DQN rely on single-step Temporal Difference learning. The target for a Q-update is based on the immediate reward and the estimated value of the next state:

$$y_t = r_t + \gamma \max_{a'} q(s_{t+1}, a'; \bar{\theta})$$

However, single-step TD learning has limitations:

1. **Myopic Updates**: It focuses only on the immediate reward, potentially missing longer-term dependencies in the reward structure.
2. **High Variance**: The value of  $q(s_{t+1}, a')$  is an estimate, and using it in updates introduces significant variance, especially when the estimate is noisy or inaccurate.

Multi-step learning addresses these issues by incorporating rewards and state values over multiple steps, enabling the agent to learn from extended sequences rather than relying solely on single transitions.

The key idea is to replace the single-step TD target with an  $n$ -step return. Instead of looking at just the immediate reward, the agent considers the sum of rewards over  $n$  steps, followed by the discounted value of the state at the  $n$ -th step:

$$G_t^{(n)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots + \gamma^{n-1} r_{t+n-1} + \gamma^n \max_{a'} q(s_{t+n}, a'; \bar{\theta})$$

The corresponding loss function becomes:

$$\mathcal{L}(\theta) = E \left[ \left( G_t^{(n)} - q(s_t, a_t; \theta) \right)^2 \right]$$

In the Deep Q-Network framework, multi-step learning modifies the way replay buffer samples are used for updates:

1. **Sequence Storage:** Instead of storing individual transitions  $(s_t, a_t, r_t, s_{t+1})$ , the replay buffer stores sequences of length  $n$ :  $(s_t, a_t, r_t, \dots, r_{t+n-1}, s_{t+n})$ .
2. **Target Calculation:** For each sequence, the  $n$ -step return  $G_t^{(n)}$  is computed as described above.
3. **Loss Function:** The Q-network is trained to minimize the loss using  $G_t^{(n)}$  as the target.

Choice of  $n$ :

- A small  $n$  captures short-term dependencies but may ignore important long-term rewards.
- A large  $n$  provides a more comprehensive signal but increases computational complexity and the risk of overestimating long-term rewards if the environment dynamics are stochastic.

## 2.5 Distributional RL

Most of RL algorithms use expected return for a given state-action pair:

$$q(s, a) = E[G_t \mid s, a]$$

where  $G_t$  represents the discounted sum of future rewards. However, this approach focuses solely on the **mean return**, ignoring the underlying distribution of possible returns. This limitation has significant consequences:

- The distribution of returns may convey crucial insights into uncertainty and risk, which are important in real-world applications (e.g., finance, robotics).
- By only modeling the mean, updates fail to capture the variability or structure of the returns, potentially leading to slower or suboptimal learning.

Distributional Reinforcement Learning [6] addresses these issues by directly approximating the **distribution of returns**  $Z(s, a)$ , rather than just its expectation.

In distributional RL, the return  $G_t$  is treated as a random variable, and its distribution  $Z(s, a)$  is propagated under the Bellman operator. The distributional Bellman equation is:

$$Z(s, a) \stackrel{D}{=} R(s, a) + \gamma Z(s', a')$$

where:

- $Z(s, a)$ : Distribution of returns for state  $s$  and action  $a$ .
- $R(s, a)$ : Immediate reward distribution.
- $\stackrel{D}{=}$ : Equality in distribution.
- $Z(s', a')$ : Distribution of future returns under the optimal policy.

The key difference from standard RL lies in modeling the full distribution  $Z(s, a)$ , rather than just its expectation  $q(s, a)$ .

Since the true return distribution is typically continuous and complex, distributional RL algorithms approximate  $Z(s, a)$  using parameterized distributions. Two common approaches are:

1. **Categorical Representation** (C51):

- The return distribution is represented as a discrete probability distribution over  $N$  fixed "atoms"  $z_i$  spaced along a value range  $[V_{\min}, V_{\max}]$ .
- The parameters are the probabilities  $p_i$  associated with each atom  $z_i$ , satisfying  $\sum_i p_i = 1$ .

The categorical representation defines  $Z(s, a)$  as:

$$Z(s, a) = \sum_{i=1}^N p_i \delta(z_i)$$

where  $\delta(z_i)$  is the Dirac delta function centered at  $z_i$ .

2. **Parametric Distribution**:

- Instead of discrete atoms,  $Z(s, a)$  is modeled using a parametric distribution (e.g., Gaussian, mixture models) with parameters learned during training.

The training process involves minimizing the divergence between the predicted and target distributions under the distributional Bellman operator. The categorical approach (C51) uses a projection step to align the target distribution with the fixed atoms:

1. Compute the target distribution:

$$Z_{\text{target}}(s, a) \sim r + \gamma Z(s', a')$$

2. Project  $Z_{\text{target}}(s, a)$  onto the fixed atom set  $\{z_i\}$ , adjusting probabilities accordingly.
3. Minimize the Kullback-Leibler divergence or another divergence metric between the predicted and target distributions.

Distributional RL can be integrated into the DQN framework with minimal architectural changes:

1. **Output Layer:** Instead of a single Q-value per action, the network outputs a vector of probabilities  $\{p_i\}$  over discrete atoms or parameters of a continuous distribution.
2. **Loss Function:** The loss is defined as the divergence between the predicted and projected target distributions, replacing the mean-squared error used in DQN.

## 2.6 Noisy Nets

Exploration is typically implemented using heuristic methods, such as  $\epsilon$ -greedy or Boltzmann exploration. These approaches introduce randomness in the action-selection process but have notable limitations:

1. Methods like  $\epsilon$ -greedy rely on predefined schedules (e.g., linearly decaying  $\epsilon$ ) that may not adapt well to varying environments.
2. Exploration noise is added at the action level, which can lead to suboptimal exploration in high-dimensional or nonstationary environments.
3. Heuristic exploration strategies do not condition noise on the agent's internal representation of the environment.

Noisy Networks [7] address these issues by integrating parameterized noise directly into the network weights, enabling the agent to learn more effective exploration strategies that adapt to the task and environment.

In Noisy Networks, the standard neural network parameters (weights and biases) are augmented with noise parameters. Specifically, each parameter  $w$  is replaced by a noisy counterpart  $w_{\text{noisy}}$ :

$$w_{\text{noisy}} = w_{\text{deterministic}} + \sigma \cdot \epsilon$$

where:

- $w_{\text{deterministic}}$ : The deterministic component of the parameter (learned during training).
- $\sigma$ : A learnable parameter controlling the scale of the noise.
- $\epsilon$ : A random variable drawn from a fixed distribution (e.g., Gaussian or uniform).

For biases, a similar transformation is applied.

The forward pass through a noisy layer is then computed as:

$$y = (w_{\text{deterministic}} + \sigma \cdot \epsilon)x + (b_{\text{deterministic}} + \sigma_b \cdot \epsilon_b)$$

Noisy Networks naturally integrate exploration into the learning process by introducing stochasticity in the network's outputs. During training:

1. Noise in the weights causes the Q-values (or policy outputs) to vary, inducing exploration.
2. The noise parameters  $\sigma$  and deterministic weights  $w_{\text{deterministic}}$  are updated using standard gradient descent. The noise's impact on learning is captured by backpropagating through the noisy components.

Noisy Networks can replace the fully connected layers in a DQN architecture with noisy layers, enabling exploration directly through parameter perturbations. This eliminates the need for  $\epsilon$ -greedy exploration. The algorithm proceeds as follows:

1. **Initialization:** Each weight and bias in the noisy layers is initialized with deterministic and noise components.
2. **Exploration:** During action selection, the network outputs are stochastic due to the noise in the parameters.
3. **Training:** Both the deterministic parameters  $w_{\text{deterministic}}$  and noise parameters  $\sigma$  are updated to minimize the TD error.

The choice of noise distribution is critical for stable training. A common approach is to use factorized Gaussian noise:

$$\epsilon_{ij} = f(\epsilon_i) \cdot f(\epsilon_j)$$

where:

- $\epsilon_i, \epsilon_j$  are independent Gaussian random variables.
- $f(x)$  is a scaling function (e.g.,  $f(x) = \text{sign}(x)\sqrt{|x|}$ ).

This factorization reduces the number of random variables needed, making the implementation more efficient.

### 3. Combination

The Rainbow algorithm integrates these six key enhancements into the Deep Q-Network framework to create a robust and efficient reinforcement learning agent. The integration is meticulously designed to ensure compatibility and maximize the synergy between the components.

#### 1. Multi-Step Learning with Distributional RL

- **Objective:** Incorporate multi-step bootstrapping into the distributional Bellman update to propagate rewards faster and improve learning efficiency.
- **Implementation:**
  - Replace the standard one-step distributional loss with a multi-step version.

- The target distribution is defined as:  $d_t^{(n)} = \left( R_t^{(n)} + \gamma^{(n)} z, p_{\theta}(S_{t+n}, a_{t+n}^*) \right)$  where  $R_t^{(n)}$  is the n-step return,  $\gamma^{(n)}$  is the cumulative discount factor,  $z$  is the distributional support, and  $a_{t+n}^*$  is the greedy action selected by the online network.
- The loss is computed using the Kullback-Leibler divergence:  $\mathcal{L}_{\text{dist}} = D_{KL}(\Phi_z d_t^{(n)} || d_t)$  where  $\Phi_z$  is the projection onto the fixed support  $z$ .

## 2. Double Q-Learning for Distributional RL

- **Objective:** Mitigate overestimation bias in the target distribution by decoupling action selection and evaluation.
- **Implementation:**
  - Use the online network to select the greedy action  $a_{t+n}^*$ :  $a_{t+n}^* = \arg \max_a Q_{\text{online}}(S_{t+n}, a)$
  - Evaluate  $a_{t+n}^*$  using the target network in the distributional Bellman update.

## 3. Prioritized Experience Replay

- **Objective:** Improve sample efficiency by prioritizing transitions with the highest learning potential.
- **Implementation:**
  - Use the KL divergence loss  $\mathcal{L}_{\text{dist}}$  to compute the priority of each transition:  $p_t \propto (\mathcal{L}_{\text{ds}})^{\omega}$  where  $\omega$  controls the sharpness of prioritization.
  - Sample transitions from the replay buffer based on their priorities and use importance sampling weights to correct for the sampling bias.

## 4. Dueling Network Architecture

- **Objective:** Separate state value and action advantage estimation to improve generalization across actions in non-critical states.
- **Implementation:**
  - Use a shared representation  $f_{\xi}(s)$  as the input to two streams:
    - **Value stream**  $v_{\eta}$  with  $N_{\text{atoms}}$  outputs.

- **Advantage stream**  $a_\psi$  with  $N_{\text{atoms}} \times N_{\text{actions}}$  outputs.
- Aggregate the streams as:  $Q(s, a) = v_\eta(f_\xi(s)) + a_\psi(f_\xi(s), a) - \frac{1}{|\mathcal{A}|} \sum_{a'} a_\psi(f_\xi(s), a')$
- Normalize the resulting distributions using a softmax layer for each action.

## 5. Distributional RL

- **Objective:** Model the full return distribution  $Z(s, a)$  instead of just the mean  $Q(s, a)$ .
- **Implementation:**
  - Represent the distribution with a categorical approximation using  $N_{\text{atoms}}$  fixed support points  $\mathbf{z}_i$  and probabilities  $p_i$ .
  - Update the probabilities using the multi-step target distribution  $d_t^{(n)}$ .

## 6. Noisy Networks

- **Objective:** Replace  $\epsilon$ -greedy exploration with adaptive noise in the network weights for state-conditional exploration.
- **Implementation:**
  - Replace linear layers with noisy layers:  $y = (b + Wx) + (b_{\text{noisy}} \odot \epsilon_b + (W_{\text{noisy}} \odot \epsilon_W)x)$  where  $\epsilon_b$  and  $\epsilon_W$  are random variables sampled from a factorized Gaussian distribution.
  - The noise parameters are learned alongside the standard weights during training.

---

The Rainbow algorithm combines these components into a single agent by aligning their mechanisms:

- **Multi-Step Learning and Distributional RL** enhance learning by propagating rewards faster and capturing the variability of returns.
- **Double Q-Learning** ensures stability in the distributional update.
- **Prioritized Replay** accelerates learning by focusing on the most informative transitions.
- **Dueling Networks** improve representational efficiency for large action spaces.
- **Noisy Networks** provide adaptive and efficient exploration.

## 4. Experimental Analysis

The Rainbow algorithm was evaluated to measure its performance on a wide range of tasks. The evaluation setup for the article is the following :

- **Benchmark:**

- Rainbow and its baselines were tested on 57 Atari 2600 games from the Arcade Learning Environment (ALE).

- **Evaluation Procedure:**

During training, the agent's performance was periodically assessed every 1 million steps using the latest policy. Each evaluation ran for 500,000 frames, with learning suspended to ensure unbiased assessment. Two evaluation regimes were employed:

- **No-op Starts:** Episodes start with up to 30 random "no-op" actions, ensuring that the agent generalizes well to varied starting conditions.
- **Human Starts:** Episodes are initialized with states sampled from human gameplay trajectories, testing how well the agent generalizes beyond its own experience.

Scores were normalized to compare across games: A score of 0% corresponds to random performance. A score of 100% corresponds to the performance of a human expert.

Aggregate metrics like the median and mean human-normalized scores were used to summarize results but the median is better suited as agents can perform far better scores than human on some games.

Only few hyperparameter tuning was made. Each extension (e.g., Noisy Nets, Prioritized Replay) retained its original hyperparameters, with minimal tuning. Adjustments were made only for sensitive parameters such as the prioritization exponent  $\omega$  (set to 0.5) and the multi-step return parameter  $n$  (set to 3 after testing  $n = 1, 3, 5$ ).

The global hyperparameters were :

- The Adam optimizer was used with a learning rate of  $\alpha = 6.25 \times 10^{-5}$ .
- The replay buffer size was 1 million transitions.
- Mini-batch size for training was set to 32.
- Exploration:
  - For Noisy Nets, exploration was handled entirely by the learned noise, eliminating the need for  $\epsilon$ -greedy.

| Parameter                                  | Value                 |
|--------------------------------------------|-----------------------|
| Min history to start learning              | 80K frames            |
| Adam learning rate                         | 0.0000625             |
| Exploration $\epsilon$                     | 0.0                   |
| Noisy Nets $\sigma_0$                      | 0.5                   |
| Target Network Period                      | 32K frames            |
| Adam $\epsilon$                            | $1.5 \times 10^{-4}$  |
| Prioritization type                        | proportional          |
| Prioritization exponent $\omega$           | 0.5                   |
| Prioritization importance sampling $\beta$ | 0.4 $\rightarrow$ 1.0 |
| Multi-step returns $n$                     | 3                     |
| Distributional atoms                       | 51                    |
| Distributional min/max values              | $[-10, 10]$           |

Table 1: Rainbow hyper-parameters from [1].



- Agents without Noisy Nets used  $\epsilon$ -greedy with a faster decay schedule than standard DQN.

### Computational Setup

- All experiments were run on a single GPU for consistency with the original DQN setup.
- Training ran for 200 million frames per agent, corresponding to approximately 10 days of wall-clock time per run.

### Analysis

Rainbow outperformed all baselines, both in terms of data efficiency (learning speed) and final performance. It achieved DQN's final performance after just 7 million frames and surpassed the best-performing baselines by 44 million frames. Median human-normalized performance across games was used as the primary evaluation metric.

*(Additional metrics included: The number of games where the agent exceeded specific thresholds of human performance (e.g., 20%, 50%, 100%, 200%, and 500%))*

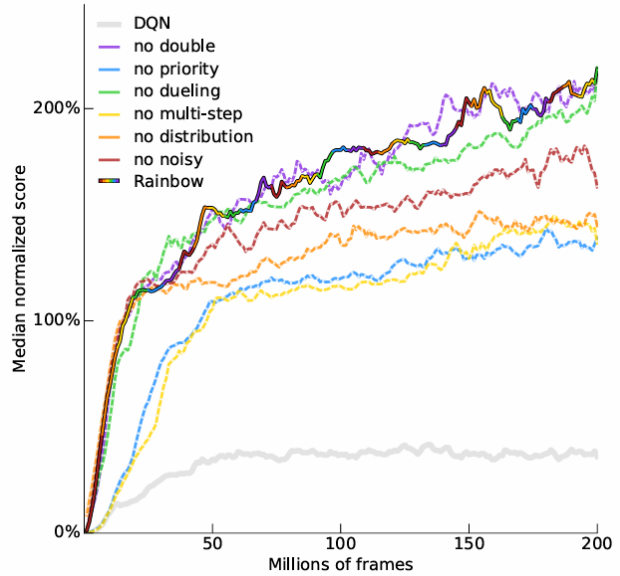


Figure 1: Median human-normalized performance across 57 Atari games from [1].

Rainbow consistently outperformed baselines across all thresholds, showing improvements in both weak and strong games.

Ablation studies were conducted to understand the contribution of each component in Rainbow. For each ablation, a single component was removed while retaining all others.

Results showed that **Prioritized Replay** and **Multi-Step Learning** had the largest impact. Removing them caused significant drops in performance across most games. These components improved both early learning and final performance, indicating their critical role in Rainbow's success.

Distributional Q-learning became increasingly important as training progressed, especially in games where the agent achieved human or superhuman performance.

Removing Noisy Nets and reverting to  $\epsilon$ -greedy reduced performance, particularly in exploration-heavy games. Noisy Nets provided state-conditional exploration, which led to better results in complex environments.

The impact of Dueling Networks varied across games. While it contributed to some high-performing games, its overall effect on the median performance was limited.

Double Q-Learning's contribution was minor in this setup, potentially due to the value clipping used in the distributional setting. Expanding the support range could make this component more impactful.

Each ablation was evaluated across all 57 games, and the results were visualized. The drop in performance for each game relative to the full Rainbow agent is represented in Figure:

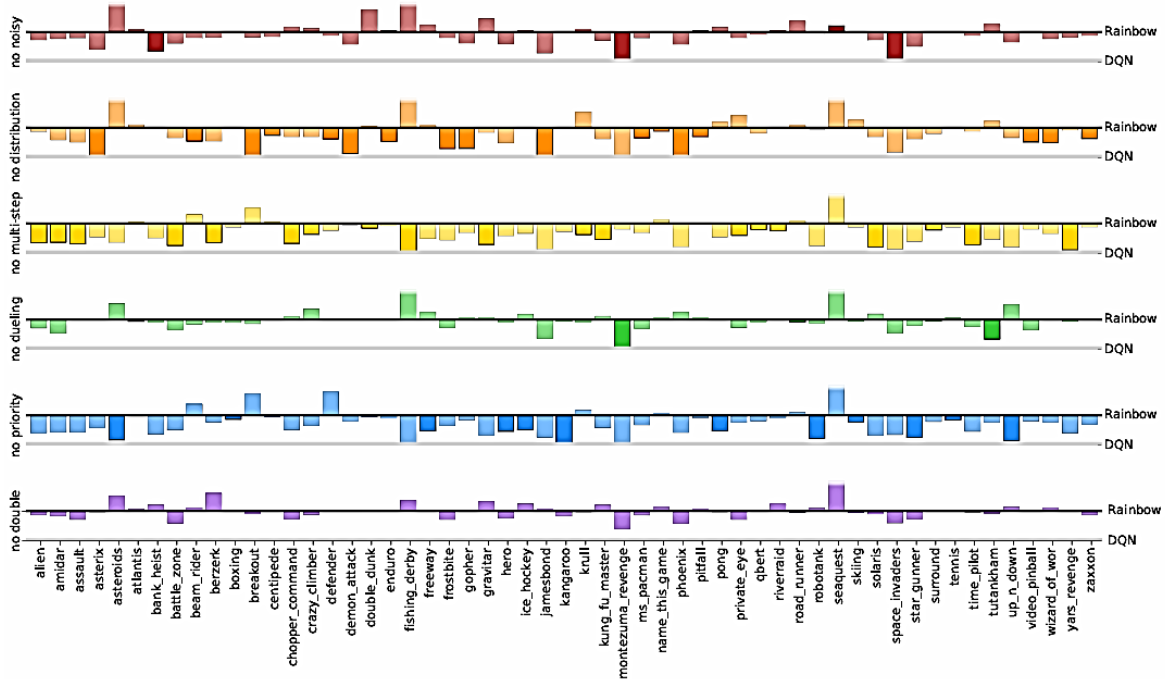


Figure 2: Performance drops of ablation agents on all 57 Atari games from [1].

The experimental methods and analysis demonstrate that Rainbow's integration of six extensions results in a highly effective RL agent. The ablation studies highlight that while some components, like Prioritized Replay and Multi-Step Learning, are universally beneficial, others, like Dueling Networks and Double Q-Learning, provide game-specific advantages. This comprehensive evaluation confirms Rainbow's state-of-the-art performance and the complementary nature of its components.

## 5. Implementation

The repository [thibautboyenval/reinforcement-learning-rainbow](https://github.com/thibautboyenval/reinforcement-learning-rainbow) contains a *rainbow.ipynb* notebook that can be executed directly on Colab. You can also directly access it here: [\[Colab\]](#). This notebook provides an object-oriented implementation of a Rainbow agent in Python. Each extension previously introduced has been implemented based on the work of [9] and [10] and

integrated jointly with a baseline DQN algorithm. The **CartPole** environment from "Classic Control" environments of Gymnasium was then used to test our agent.

The **CartPole** environment in Gymnasium is a classic control problem used for testing reinforcement learning algorithms. The task is to balance a pole attached to a cart by applying forces to move the cart left or right along a one-dimensional track.

#### Environment Details:

- **Objective:** Prevent the pole from falling over by keeping it balanced upright for as long as possible.
- **Action Space:** Discrete
  - ``0``: Move the cart to the left.
  - ``1``: Move the cart to the right.
- **Observation Space:** A continuous 4-dimensional space representing:
  - Cart position (``x``): Horizontal position of the cart.
  - Cart velocity (``x_dot``): Speed of the cart along the track.
  - Pole angle (``theta``): Angle of the pole relative to vertical.
  - Pole angular velocity (``theta_dot``): Rate of change of the pole's angle.
- **Reward:**
  - `+1` for every time step the pole remains balanced.
  - The goal is to maximize the cumulative reward.
- **Termination Conditions:**
  - The pole angle exceeds  $\pm 12$  degrees from vertical.
  - The cart's position moves more than  $\pm 2.4$  units from the center of the track.
  - Episode length exceeds 500 time steps (depending on configuration).
- **Physics:** The environment simulates a simplified physical system based on a frictionless track and pole dynamics.

## 6. Improvements for Rainbow

This work focuses on value-based methods in the Q-learning family, excluding policy-based and actor-critic approaches. Sequential data techniques like Optimality Tightening and eligibility traces enhance learning efficiency but require more computation than Rainbow's multi-step targets. Episodic control improves early learning using episodic memory to replicate successful actions, while exploration strategies such as Bootstrapped DQN, intrinsic motivation, and count-based methods remain promising for future research.

Alternative architectures, like asynchronous methods (e.g., A3C or Gorila), can accelerate learning in wall-clock time but are less data-efficient. Hierarchical RL (e.g., h-DQN and Feudal Networks) and improved state representation via auxiliary tasks, supervised predictions, or successor features are also valuable directions. Domain modifications like reward clipping and

action repetition used in Rainbow evaluations might be replaced by techniques such as Pop-Art normalization or fine-grained action repetition, offering more dynamic learning strategies.

## References

- [1] M. Hessel, J. Modayil, H. van Hasselt, T. Schaul, G. Ostrovski, W. Dabney, D. Horgan, B. Piot, M. Azar, and D. Silver, "Rainbow: Combining Improvements in Deep Reinforcement Learning," *arXiv preprint*, arXiv:1710.02298, 2017.
- [2] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
- [3] H. van Hasselt, A. Guez, and D. Silver, "Deep Reinforcement Learning with Double Q-learning," *arXiv preprint*, arXiv:1509.06461, 2015.
- [4] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized Experience Replay," *arXiv preprint*, arXiv:1511.05952, 2015.
- [5] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, and N. de Freitas, "Dueling Network Architectures for Deep Reinforcement Learning," *arXiv preprint*, arXiv:1511.06581, 2015.
- [6] M. G. Bellemare, W. Dabney, and R. Munos, "A Distributional Perspective on Reinforcement Learning," *arXiv preprint*, arXiv:1707.06887, 2017.
- [7] M. Fortunato, M. Azar, B. Piot, J. Menick, I. Osband, A. Graves, V. Mnih, R. Munos, D. Hassabis, O. Pietquin, et al., "Noisy Networks for Exploration," *arXiv preprint*, arXiv:1706.10295, 2017.
- [8] R. S. Sutton, "Learning to predict by the methods of temporal differences," *Machine Learning*, vol. 3, no. 1, pp. 9–44, 1988.
- [9] Higgsfield. *RL Adventure*. GitHub repository. Available at: <https://github.com/higgsfield/RL-Adventure/tree/master>.
- [10] Kaixhin. *Rainbow: A PyTorch Implementation of Rainbow DQN*. GitHub repository. Available at: <https://github.com/Kaixhin/Rainbow/tree/master>.